

Introducción a la Programación

Clase 02: Secuenciales y Condicionales

Daniela Opitz
daniela.opitz@usm.cl



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase de Hoy

- Repaso secuenciales
- Operadores de comparación (`==`, `!=`, `<`, `<=`, `=>`, `>`)
- Operadores de lógica (or, and, not)
- Condicionales
- Actividades

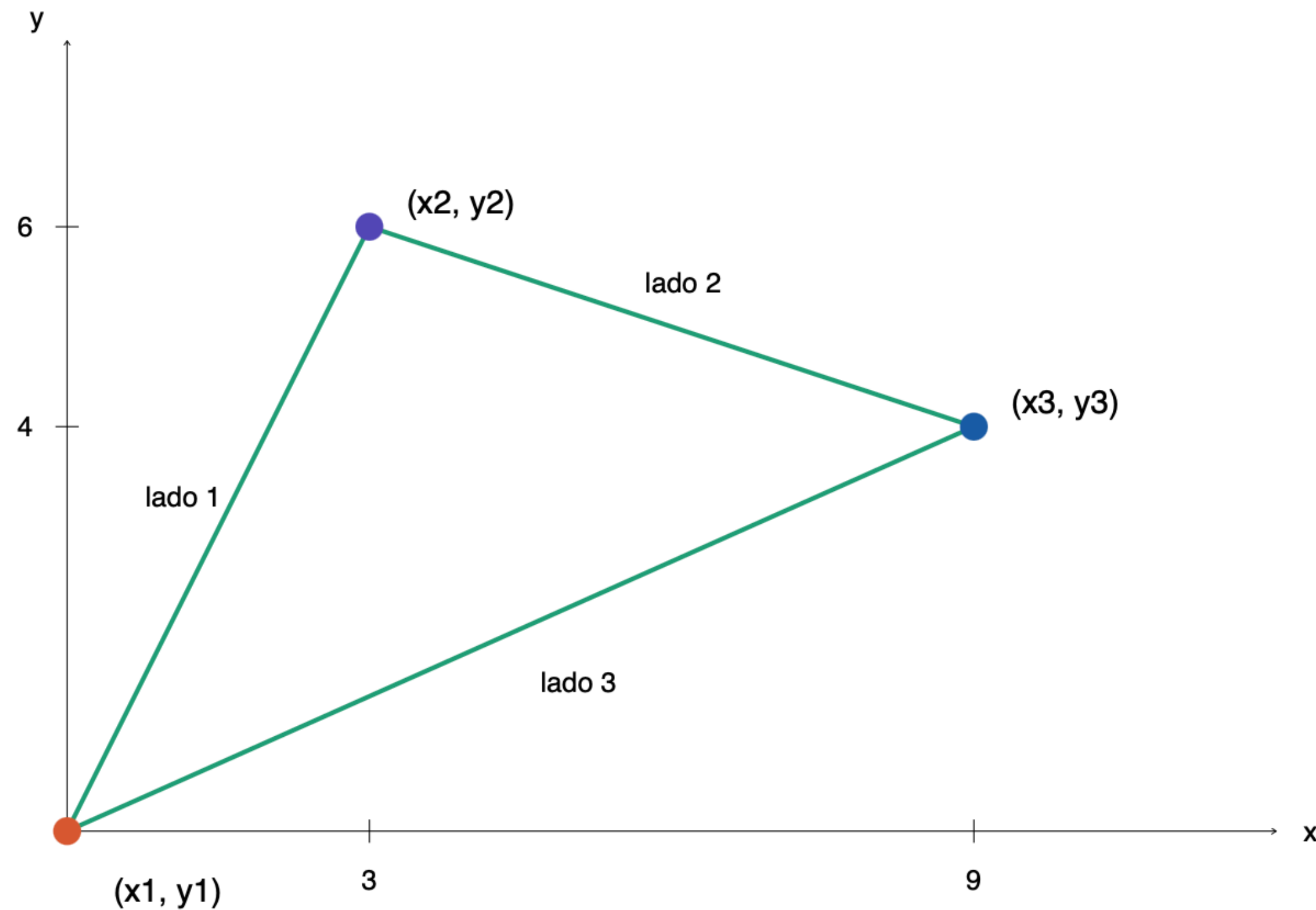
Actividad 0

Escriba un programa que calcule la distancia entre dos puntos de los que conocemos sus coordenadas. Recuerde que la fórmula para calcular la distancia entre (x_1, y_1) y (x_2, y_2) es:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Actividad 0

Tenemos un triángulo del cual conocemos las coordenadas de sus 3 vértices. Escriba un programa que calcule su perímetro. Pida a un usuario sus coordenadas.



Operadores de Comparación

- Permiten verificar si variables cumplen algunas reglas básicas.
- Devuelven un valor de tipo bool (**True**, **False**)

Sintáxis	Operador	Ejemplo	Resultado
a < b	menor que	2 < 5	True
a <= b	menor o igual	2 <= 2	True
a >= b	mayor o igual	2 >= 32	False
a > b	mayor	0 > -1	True
a == b	igual	1 == -1	False
a != b	distinto	1 != -1	True

```
a = 2
b = -1
print('a < b:', a < b)
print('a > b:', a > b)
print('a <= b:', a <= b)
print('a >= b:', a >= b)
print('a == b:', a == b)
print('a != b:', a != b)
```



```
a < b: False
a > b: True
a <= b: False
a >= b: True
a == b: False
a != b: True
```

Operadores Lógicos

- Los operadores lógicos proporcionan un resultado de acuerdo al cumplimiento o no de una cierta condición.
- Operadores lógicos en Python (**or**, **and**, **not**)
- El resultado de **or** es verdadero cuando al menos una de las expresiones lo es.
- El resultado de **and** es verdadero solo cuando ambas expresiones lo son.
- El operador **not** invierte el valor del operando.

Sintáxis	Operador
not	Negación
and	Conjunción
or	Disyunción

Operadores Lógicos

Tabla de Verdad

a	b	not a	a or b	a and b
False	False	True	False	False
False	True	True	True	False
True	False	False	True	False
True	True	False	True	True

```
a = True
b = False
c = a or b
print('not a =', not a)
print('a or b =', a or b)
print('a and b =', a and b)
print('c =', c)
```



```
not a = False
a or b = True
a and b = False
c = True
```

Prioridad	1	1	2	3	4
Operador	==	!=	not	and	or

—————→ **== y != tienen la misma prioridad**

Prioridades

Prioridad	1	1	2	3	4
Operador	==	!=	not	and	or

== y != tienen la misma prioridad

```
>>x = 5
>>print(not x == 5 or x != 3)
```

Razonamiento

```
print(not True or True)
print(False or True)
print(False)
```

¿Resultado?

Indentación

- Anglicismo proveniente de **indentation**. Es un espacio vacío al comienzo de una línea para marcar bloque de código.

```
Esta linea esta indentada por 1 espacio.  
  Esta linea esta indentada por 2 espacios.  
    Esta linea esta indentada por 3 espacios.
```

- Puede usar espacios o tabs (`\t`) como indentación pero sin mezclarlos. **Sea consistente!**
- Se estila usar cuatro espacios por nivel de indentación. En los editores Python, como Spyder, esto lo realiza el editor.

Condicionales

- Palabras claves: **if**, **else**
- Permiten condicionar la ejecución de uno o varios bloques de sentencias al cumplirse una o varias condiciones.
- **Sintaxis:**

```
if condición:  
    aquí van las instrucciones que se ejecutan si la condición es cierta (True)  
  
else:  
    aquí van las instrucciones que se ejecutan cuando la condición es falsa (False)
```

```
1 a = 50  
2 b = 10  
3 if a > b:  
4     print('a es mayor que b')
```

if

Aquí las instrucciones 1 2 3 se ejecutarán si la **condición** es verdadera.

Las instrucciones 4 y 5 siempre se ejecutarán a continuación de **if**, **independiente** del resultado de la evaluación de la condición. El término de la indentación marca el fin de un bloque de código.

indentación

```
if condición:
    instrucción 1
    instrucción 2
    instrucción 3
#fin del if
instrucción4
instrucción5
```

condiciones

<	>	<=	>=	==	!=
menor	mayor	menor igual	mayor igual	igual	distinto

Condicionales

- ¿Cómo resuelvo un problema que involucra condicionales?
 1. Descomponer el problema en etapas
 2. Para cada etapa reconocer qué reglas se deben cumplir
 3. Ejecutar acciones por cada regla

```
1 a = 5
2 b = 10
3 if a > b:
4     # cuando la condición se cumple
5     print('a es mayor que b')
6 else:
7     # cuando la condición no se cumple
8     print('a es menor o igual que b')
```

Actividad 1

- Escribe un programa que recibe un numero entero por teclado e imprime si el número es par o no lo es.

Actividad 2

En una librería, cada libro cuesta **12.000 pesos**. Si un cliente compra **más de 3 libros**, recibe un **10% de descuento** en el total.

Escriba un programa que:

1. Pida la cantidad de libros comprados.
2. Calcule el precio total.
3. Si corresponde, aplique el descuento.
4. Muestre el total a pagar.

Actividad 3

Crea un programa que calcule la solución de una ecuación de segundo grado de la forma:

$$ax^2 + bx + c = 0$$

Asume que el usuario entrega el valor de a , b y c . Considere **sólo** soluciones en los reales e informe al usuario en caso de que la solución no tenga solución en ellos. Recuerda que la raíz se puede calcular con:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Actividad 4

Un estudiante rinde dos certámenes y tiene un porcentaje de asistencia a clases.

- Cada certamen se califica de 0 a 100 puntos.
- Para aprobar, el promedio de los dos certámenes debe ser mayor o igual a 55 y la asistencia debe ser de al menos 75%.

Escriba un programa que:

- Solicite las dos notas y el porcentaje de asistencia.
- Muestre:
 - "**Aprobado**" si cumple ambas condiciones.
 - "**Reprobado por nota**" si el promedio es menor a 55.
 - "**Reprobado por asistencia**" si el promedio cumple, pero la asistencia es menor a 75%.